

Chapter - 14

Errors & Exceptions

No matter how smart we are, errors are our constant companions with practice. We keep getting better at finding & correcting them.

There are three types of errors in Java

- 1) Syntax Error
- 2) Logical Error
- 3) Runtime Error.

Syntax Error

When compiler find something wrong with our program. It throws a syntax error.

`int a = 9` → no semicolon, syntax error
`a = a + 3;`

`d = 4;` → Variable not declared

Logical Errors

A logical error or a bug occurs when a Program compiles and runs but does ~~not~~ the wrong thing.

2017 → message delivered wrongly.

→ wrong time of chats being displayed

Runtime Errors

Java may sometimes encounter an error while the program is running. These are also called exception.

There are encountered due to circumstances like bad input or resource constraints

Ex: user supplies 5 + 8 to a program which add two numbers.

Syntax errors and logical errors are encountered by the programmer whereas Runtime errors are encountered by user.

Examples:

```
public class ErrorDemo
```

```
{
    public static void main (String[] args)
    {
```

```
        // SYNTAX ERROR DEMO
```

```
        int a = 5 // ERROR: NO SEMICOLON
```

```
        b = 10; // ERROR: B NOT DECLARED
```

```
        // LOGICAL ERROR DEMO
```

```
        // WRITE A PROGRAM TO PRINT ALL PRIME NO
```

```
        System.out.println(2);
```

```
        for (int i = 0; i < 5; i++)
```

```
{
```

```
            System.out.println(2 * i + 1);
```

// RUNTIME ERROR

```
int k;
Scanner sc = new Scanner(System.in);
k = sc.nextInt();
System.out.println("Integer part of 1000 divided by k is " + 1000/k);
}
```

Exceptions in Java

An Exception is an event that occurs when a program is executed disrupting the normal flow of instructions. There are mainly two types of Exception in Java:

- 1) checked Exception → Compiled time exceptions (Handled by compiler)
- 2) unchecked Exception → Runtime exception.

Commonly Occurring Exceptions

- 1) NullPointerException
- 2) Arithmetic Exception
- 3) Array Out of Bound Exception
- 4) Illegal Argument Exception
- 5) Number Format Exception

Try Catch Block in Java

```

public class TRY_EXCEPTION
{
    public static void main (String [] args)
    {
        int a = 6000;
        int b = 0;
        try {
            int c = a / b;
            System.out.println ("The result is " + c);
        }
        catch (Exception e)
        {
            System.out.println ("We failed to divide. Reasons:");
            System.out.println (e);
        }
        System.out.println ("End of the Program");
    }
}
  
```

Handling Specific Exceptions

```

10 public class HANDLING_EXCEPTION
11 {
12     public static void main (String [] args)
13     {
14         int [] marks = new int [3];
15         marks [0] = 7 ;
16         marks [1] = 56 ;
17         marks [2] = 60 ;
18         Scanner sc = new Scanner (System.in);
19         System.out.println ("Enter the array index");
20         int ind = sc.nextInt ();
21         System.out.println ("Enter the number you want divide the value with");
22         int number = sc.nextInt ();
23         try
24         {
25             System.out.println ("The value of array index Entered is: " + marks [ind]);
26             System.out.println ("The Value of array value / number is: " + marks [ind] / number);
27         }
28         catch (ArithmeticException e)
29         {
30             System.out.println ("Arithmetic Exception occurred");
31             System.out.println (e);
32         }
33     }
34 }
  
```

catch (ArrayIndexOutOfBoundsException e)

{
 System.out.println("ArrayIndexOutOfBoundsException occurred");

 System.out.println(e);

}
catch (Exception e)

{
 System.out.println("Some Exception occurred");
 System.out.println(e);

SUN 07

Nested Try-Catch in Java

```

10 public class NESTED-TRY
11 {
12     public static void main (String [] args)
13     {
14         int [] marks = new int [3];
15         marks [0] = 7;
16         marks [1] = 56;
17         marks [2] = 6;
18         Scanner sc = new Scanner (System.in);
19         System.out.println ("Enter the value of index");
20         int ind = sc.nextInt ();
21         try {
22             System.out.println ("welcome to the Java");
23             try {
24                 System.out.println (marks [ind]);
25             }
26             catch (ArrayIndexOutOfBoundsException)
27             {
28                 System.out.println ("Sorry, this index
29                                     does not exist");
30                 System.out.println ("Exception in level 2");
31             }
32             catch (Exception e)
33             {
34                 System.out.println ("Exception in level 1");
35             }
36         }
37     }
38 }

```

Java Exception class

```
public class EXCEPTION_CLASS
```

```
{
    public static void main (String [] args)
```

```
{
```

```
    int a;
```

```
    Scanner sc = new Scanner (System.in)
```

```
    a = sc.nextInt();
```

```
    if (a < 9) {
```

```
        try {
```

```
            throw new MyException();
```

```
            throw new ArithmeticException("This is an Exception");
```

```
        }
```

```
    } catch (Exception e) {
```

```
        System.out.println(e.getMessage());
```

```
        System.out.println(e.toString());
```

```
        e.printStackTrace();
```

```
        System.out.println("Finished");
```

```
    }
```

```
    System.out.println("Yes Finished");
```

```
}
```

```
}
```

```
}
```

```
class MyException extends Exception {
```

```
    @Override
```

```
    public String toString() {
```

```
        return super.toString() + "I am toString()";
```

```
}
```

9 @Override

public String getMessage() {

return super.getMessage() + "I am get Message!"

11 }

3

PRACTICE-SET - 14

1) Write a Java Program to demonstrate Syntax, logical & runtime errors.

2) Write a Java Program that prints "Hatta" during Arithmetic Exception and "Hette" during an illegal argument Exception.

3) Write a Program that allows you to keep accessing an array until a valid index is given. if man returns exceeds 5 print "error".

4) Modify Program in Q3 to throw a custom Exception if man returns are reached.

5) Wrap the Program in Q3 inside a method which throw your custom Exception.